# Prototype

Prototype · Clone weekly-meeting doc templates per team · Build with Waqar

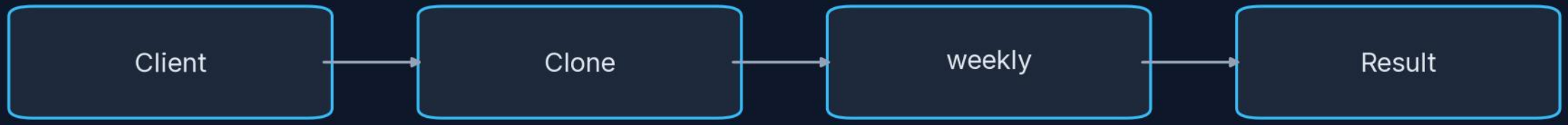
# Today's Hook

- Problem: Clone weekly-meeting doc templates per team
- Pattern: Prototype — earned from the pain, not trivia.
- Runnable main on JDK 25 with captured stdout.
- Naive → solution → failure paths → trade-offs.

# The Problem

- Clone weekly-meeting doc templates per team
- Naive design shows the cost clearly.
- Pattern isolates the force behind a clean boundary.
- Tests cover happy path and injected failures.

# Structure




Prototype: Clone weekly-meeting doc templates per team

# Real-World Scenario

- Running example: Clone weekly-meeting doc templates per team
- Same names in naive code, refactor, and Demo output.
- No shared StreamForge framing unless it fits.
- Product language first; pattern vocabulary second.

# Naive Code

```java
package com.buildwithwaqar.patterns.gof.creational.prototype;

import com.buildwithwaqar.patterns.gof.creational.prototype.DocumentTemplate;
import java.util.List;

/** Problem: clone a weekly sync template per team without rebuilding sections. */
public final class UseCaseDemo {
    public static void main(String[] args) {
        System.out.println("=== Prototype problem: clone meeting template ===");
        var proto = new DocumentTemplate("Weekly Sync", List.of("Agenda", "Notes"));
        var teamA = proto.clone();
        teamA.setTitle("Weekly Sync — Platform");
        teamA.sections().add("Action Items");
        System.out.println("proto=" + proto.title() + proto.sections());
        System.out.println("clone=" + teamA.title() + teamA.sections());
    }
}
```

# Why Naive Hurts

- Call sites couple to the wrong shape or leak on partial failure.
- Parameter drilling / duplicated envelopes / missing rollback.
- Hard to unit-test without the real dependency.
- New variants require shotgun edits.

# Solution

```java
package com.buildwithwaqar.patterns.gof.creational.prototype;

import java.util.ArrayList;
import java.util.List;

/** Prototype: clone a preconfigured object instead of rebuilding from scratch. */
public final class DocumentTemplate implements Cloneable {
    private String title;
    private final List<String> sections;

    public DocumentTemplate(String title, List<String> sections) {
        this.title = title;
        this.sections = new ArrayList<>(sections);
    }

    public String title() { return title; }
    public void setTitle(String title) { this.title = title; }
    public List<String> sections() { return sections; }

    /** Deep copy so mutating the clone does not affect the prototype. */
    @Override
    public DocumentTemplate clone() {
        return new DocumentTemplate(title, sections);
    }
```

# Solution (2)

```
}

/** Modern: records are naturally copyable via canonical constructor / with-style helpers. */
record SlideDeck(String title, List<String> slides) {
    SlideDeck {
        slides = List.copyOf(slides);
    }

    SlideDeck customized(String newTitle) {
        return new SlideDeck(newTitle, slides);
    }
}
```

# Runnable main

```java
package com.buildwithwaqar.patterns.gof.creational.prototype;

import com.buildwithwaqar.patterns.gof.creational.prototype.DocumentTemplate;
import java.util.List;

/** Problem: clone a weekly sync template per team without rebuilding sections. */
public final class UseCaseDemo {
    public static void main(String[] args) {
        System.out.println("=== Prototype problem: clone meeting template ===");
        var proto = new DocumentTemplate("Weekly Sync", List.of("Agenda", "Notes"));
        var teamA = proto.clone();
        teamA.setTitle("Weekly Sync — Platform");
        teamA.sections().add("Action Items");
        System.out.println("proto=" + proto.title() + proto.sections());
        System.out.println("clone=" + teamA.title() + teamA.sections());
    }
}
```

# Walkthrough Output

```
=== Prototype problem: clone meeting template ===
proto=Weekly Sync[Agenda, Notes]
clone=Weekly Sync — Platform[Agenda, Notes, Action Items]
```

# Step-by-Step Trace

- Each stdout line maps to a stage above.
- Re-run the main class locally on JDK 25.
- Failure-path lines prove compensations / rejects / missing files.
- Pause and predict the next line before revealing.

# Modern Java 25 / Zazr

```
// Java 25 / Zazr direction
// records + sealed + switch
// inject Env / collaborators at the edge
// virtual threads for blocking fan-out
```

# Trade-offs

- Clarity vs ceremony — refuse five types for two lines.
- Teaching demos omit production metrics/idempotency keys.
- Prefer simpler language features when they erase the force.
- Know the twin pattern often confused with Prototype.

# Related Patterns

- Contrast Prototype with its neighbor pattern.
- Name the roles from memory in sixty seconds.
- Point to the season map for what comes next.
- Link the GitHub episode folder in the description.

# Recap

- Problem → naive → Prototype → Demo stdout.
- Failure paths are part of the lesson.
- Teach when NOT to use it.
- Practice: break a test, fix it, re-run the main.